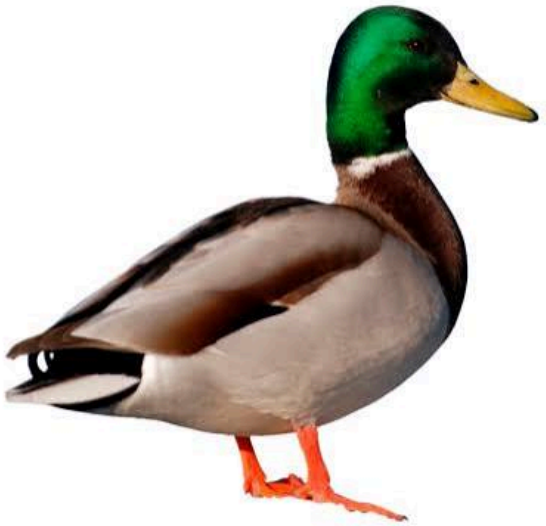
Ciência de Dados Aplicada ao Direito I

Aula 22

Do canabidiol ao Tema 1.417



Prof. Bruno da Cunha de Oliveira - 2026/2

Onde estamos

Insper

O que já respondemos à empresa de canabidiol?

A empresa fez três perguntas sobre as 300 sentenças do TJSP: contra quem se judicializa mais, quem é mais acionado em cada grupo, e quem ganha.

Para cada uma: como respondemos, e o que ficou faltando?

Três perguntas, três regras, três ressalvas

RESPOSTA

Pergunta	Como respondemos	O que faltou
Público ou privado?	regex do réu e regra por palavras: 196 de 300 contra o Estado	12 sem réu; "canabidiol" puxa o SUS
Quem é mais acionado?	contar o réu	a mesma Fazenda escrita de três jeitos
Quem ganha?	regex do dispositivo	40 de 300 sem classe

Outra maneira: utilizando Inteligência Artificial

Quem mais poderia preencher as três colunas?

Em vez de descrever o padrão do texto, dá para pedir a alguém que leia cada sentença e responda: quem é o réu, se é público ou privado, qual foi o resultado.

Ler 300 à mão leva um dia. Quem mais lê texto e responde em formato fixo?

Um modelo de linguagem

RESPOSTA

Um modelo de linguagem lê a sentença e preenche o formulário. Chamado de dentro do programa, ele faz isso para cada linha da tabela.

Para isso precisamos de três coisas: uma chave de acesso a um serviço que roda o modelo, um formulário com as perguntas, e a biblioteca que liga os dois à tabela.

Como o modelo escreve

O texto vira tokens

Animação no navegador: `animacao-tokens.html`, cenário `quebrar`.

A próxima palavra de uma ementa

```
EMENTA: Apelação cível. Plano de saúde.
Negativa de cobertura. Recurso ...
```

Que palavra vem depois de "Recurso"? Deem também uma chance, em porcentagem, para o seu palpite.

Um token de cada vez

Animação no navegador: `animacao-tokens.html`, cenário `gerar`.

Um preditor do próximo token

Dado o texto até aqui, o modelo dá uma chance a cada token possível e escolhe o próximo. Depois repete.

O treino fica cada vez mais sofisticado, mas o que o modelo faz ao responder é isso. Preencher um formulário é escrever, token a token, um texto no formato que o formulário pede.

O computador por trás do modelo

O gpt-oss-120b roda no notebook de vocês?

O modelo que vamos usar pela Groq tem 117 bilhões de parâmetros: os números que ele multiplica para prever cada token.

Só os pesos ocupam uns 61 GB. Um notebook comum tem de 8 a 16 GB de memória. O que é preciso para rodar o modelo?

Uma placa de vídeo de dezenas de milhares de dólares

RESPOSTA

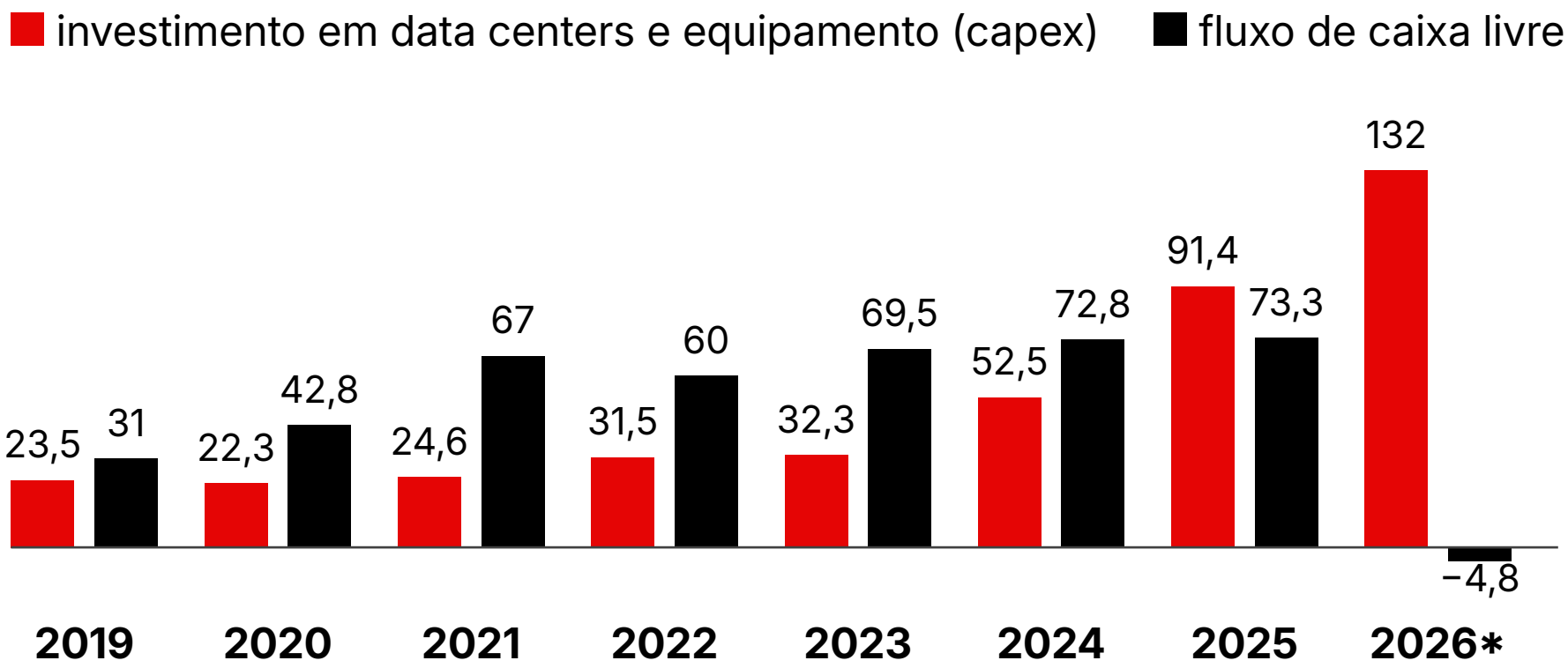
O que	Quanto
GPU que comporta o modelo	uma NVIDIA H100, de 80 GB
comprar uma H100	de US$ 25 mil a 40 mil (em 2024)
um servidor com 8 H100	de US$ 300 mil a 500 mil
alugar uma H100	uns US$ 4 por hora
velocidade de uma conversa	de 170 a 500 tokens por segundo

O que é um data center

Um galpão cheio de servidores como esse, com energia e resfriamento para todos ao mesmo tempo. Cada servidor atende milhares de pedidos por vez.

Data center	Tamanho
Microsoft, Wisconsin	centenas de milhares de GPUs em 127 hectares, por dezenas de bilhões de dólares
OpenAI, Texas	até 400 mil chips e 1,2 GW, a energia de uma cidade grande (reportagem de 2025)

Google: o gasto com data centers come o caixa



Alphabet, em US$ bilhões; *2026, de janeiro a setembro. Fluxo de caixa livre: caixa das operações menos capex. Fonte: SEC.

Como acessar um modelo

Três jeitos de chegar a um modelo

Jeito	Como paga
API de uma empresa: Groq, OpenAI, Anthropic, Google	por token, com uma chave
assinatura do Claude ou do ChatGPT, pelo Claude Code ou pelo Codex	a cota da assinatura
o modelo aberto num computador próprio	a GPU e a energia

Hoje: a API da Groq, com a conta gratuita. O dataframeit também aceita o Claude Code e o Codex.

Criar a conta e a chave na Groq

#	O que fazer
1	Abrir `console.groq.com` e entrar com a conta do Google, do GitHub ou um e-mail
2	No menu, **API Keys**, e depois **Create API Key**; dar um nome, como `cdad1`
3	Copiar a chave agora: ela começa com `gsk_`

A chave diz à Groq quem está pedindo: pedido, cota e cobrança caem na conta de quem a criou. Não mandem a ninguém, nem para mim.

Onde guardar a chave

O jeito mais rápido é colar a chave no próprio programa:

```
chave = "gsk_..."
```

O programa funciona. O que pode dar errado?

A chave escrita no código

Animação no navegador: `animacao-api-llm.html`, cenário `codigo`.

A chave vai junto com o arquivo

RESPOSTA

O arquivo vai para o repositório, e a chave vai junto: no repositório da fase 2, quem o abrir usa a conta de vocês. Apagar a linha e enviar de novo não resolve, porque a versão anterior continua no histórico.

Chave que vazou se apaga no painel da Groq, em **API Keys**, e se troca por outra.

A chave no arquivo .env

#	O que fazer, na pasta `cdad1`
1	No terminal do VS Code: `uv add python-dotenv`
2	Criar o arquivo `.env`, com uma linha só: `GROQ_API_KEY=` e a sua chave colada depois do =
3	Abrir o arquivo `.gitignore` e acrescentar uma linha com `.env`

O `.gitignore` lista o que o Git não registra: o `.env` fica no computador e nunca vai para o GitHub.

Conferir a chave

```python
import os
from dotenv import load_dotenv

load_dotenv()
chave = os.environ["GROQ_API_KEY"]
print(chave[:4])
```

No caderno da aula 22, passo 1. O que aparece? E se o `.env` não existisse?

os.environ lê o ambiente

RESPOSTA

```
gsk_
```

O `load_dotenv()` leva o que está no `.env` para o ambiente, e `os.environ` é um dicionário com ele. Sem o arquivo, a chave não existe:

```
KeyError: 'GROQ_API_KEY'
```

Só os quatro primeiros caracteres vão para a tela: a chave inteira nunca aparece num print.

O modelo preenche a tabela

Ler o README de uma biblioteca

A biblioteca é o dataframeit, no GitHub. O README de uma biblioteca costuma ter o que basta para a primeira chamada: como instalar, um exemplo mínimo e a lista de argumentos.

```
uv add "dataframeit[groq]"
```

Vamos ler juntos o exemplo do README e adaptá-lo às nossas três perguntas.

```python
from pydantic import BaseModel
from typing import Literal
import pandas as pd
from dataframeit import dataframeit

# 1. Defina o que extrair
class Sentimento(BaseModel):
    sentimento: Literal['positivo', 'negativo', 'neutro']
    confianca: Literal['alta', 'media', 'baixa']

# 2. Seus dados
df = pd.DataFrame({
    'texto': [
        'Produto excelente! Superou expectativas.',
        'Péssimo atendimento, nunca mais compro.',
        'Entrega ok, produto mediano.'
    ]
})

# 3. Processe!
resultado = dataframeit(df, Sentimento, "Analise o sentimento do texto.")
print(resultado)
```

Os argumentos, na documentação

Parâmetro	Tipo	Obrigatório	Descrição
`data`	DataFrame, Series, list, dict	Sim	Dados contendo os textos a processar
`questions`	Pydantic BaseModel	Sim	Modelo Pydantic definindo campos a extrair (`perguntas` é aceito como nome obsoleto do mesmo parâmetro)
`prompt`	str	Sim	Template do prompt. Use `{texto}` para posicionar o texto
`text_column`	str	Não	Nome da coluna com textos. Se `None`, tenta `texto`, `text`, `decisao`, `content`, `content_text` na ordem (ou a única coluna, se o DataFrame tiver apenas uma). Sem candidato e com várias colunas, levanta `ValueError`

... e o modelo que responde

Modelo

Parâmetro	Tipo	Padrão	Descrição
`model`	str | None	`None`	Nome do modelo LLM; `None` usa o modelo padrão do provider, listado em Provedores
`provider`	str	`'openai'`	Identificador do provider; `claude_code` e `codex` usam os SDKs oficiais em vez de LangChain (ver Provedores)
`api_key`	str	`None`	API key (usa env var se None); não aceito com `provider='codex'` e ignorado com `provider='claude_code'`
`model_kwargs`	dict	`None`	Parâmetros extras; com `claude_code`, só `max_turns`, `max_budget_usd` e `effort` são lidos, e o resto é ignorado; com `codex`, apenas `effort` e `timeout` são aceitos

Como adaptar o exemplo às dez sentenças?

O exemplo classifica sentimento em três frases, com o provedor padrão. Nós temos as dez primeiras sentenças, o nosso formulário e a Groq.

O que trocamos na chamada do README, e que argumento acrescentamos?

Uma chamada para dez sentenças

RESPOSTA

```
from dataframeit import dataframeit
dez = decisoes.head(10)
tabela = dataframeit(dez, Formulario,
    "Leia a sentença e preencha os campos: {texto}",
    provider="groq", text_column="texto")
print(tabela[["requerido", "polo_passivo", "resultado"]])
```

Trocamos dados, formulário e pedido; entram `provider` e `text_column`.

Primeiro formulário: tudo em texto

```
class Formulario(BaseModel):
    requerido: str
    polo_passivo: str
    resultado: str
```

Um campo por linha: nome, dois-pontos e tipo; `str` é texto livre. A classe herda de `BaseModel` o trabalho de conferir cada resposta. O pedido: "me diga quem é o requerido, se ele é público ou privado, e qual foi o resultado".

Rodem o primeiro formulário

No caderno, passo 4: o formulário só com texto, nas dez. Na conta gratuita, uns três minutos.

Enquanto roda: o que vocês esperam ver na coluna `resultado`?

O que volta nas dez?

Com esse formulário, o modelo preenche as três colunas das dez primeiras sentenças.

Dá para contar quantas são procedentes direto na coluna `resultado`?

Dez respostas sem padrão

RESPOSTA

requerido	polo_passivo	resultado
Público	Fazenda Pública... e o Município de Marília	Procedente – condenação solidária...
UNIMED DE GUARULHOS... (entidade privada de saúde)	Requerido	Ação parcialmente procedente: a liminar...

Fechar as opções: `Literal`

```python
from typing import Literal
from pydantic import BaseModel
class Formulario(BaseModel):
    requerido: str
    polo_passivo: Literal["público", "privado"]
    resultado: Literal["procedente",
                       "parcialmente procedente",
                       "improcedente",
                       "extinto sem julgamento do mérito"]
```

O que `Literal` muda

`Literal` fecha a lista de valores que o modelo pode devolver, e a classe recusa o que estiver fora dela. "Público" e "público" deixam de ser duas respostas.

Com a lista fechada, duas rodadas do mesmo modelo, ou dois modelos diferentes, ficam mais próximos: a resposta passa a ser replicável.

Rodem o formulário fechado

No caderno, passos 5 e 6: o formulário com `Literal`, na mesma chamada. Na conta gratuita, uns três minutos.

Cada rodada com um nome novo, `dados2 = decisoes.head(10)`: o dataframeit escreve as respostas na tabela que recebe, e a segunda chamada sobre ela pula as linhas já respondidas.

Deu certo quando `tabela` tem dez linhas e três colunas novas, cada uma com um valor que a classe aceitou.

Dez linhas vazias

```
ERRO DE AUTENTICAÇÃO - Chave de API inválida
...
  requerido polo_passivo resultado
0      None         None      None
1      None         None      None
2      None         None      None
```

O programa não para: cada linha que falha volta vazia, e o aviso fica acima da tabela. Quando é a chave, conferir o `.env` (o nome é `GROQ_API_KEY`, sem espaço em volta do =) e o passo 1.

Por trás do dataframeit

Insper

O que o dataframeit fez com as dez?

Uma chamada, dez linhas preenchidas. Com o que vocês já sabem de Python, como escreveriam isso sem a biblioteca?

Um `for`, uma chamada por linha

RESPOSTA

```
respostas = []
for texto in dez["texto"]:
    pedido = prompt.replace("{texto}", texto)
    resposta = modelo(pedido)      # vai à Groq e volta
    respostas.append(Formulario(resposta))  # confere
# e põe cada campo numa coluna da tabela
```

É um esquema, não o código da biblioteca: `modelo` aqui é a chamada à Groq.

O que vem além do `for`

O que a biblioteca faz	Onde aparece
conta os tokens de cada linha	`_input_tokens`, `_output_tokens`, `_reasoning_tokens`
anota a linha que falhou e por quê	`_dataframeit_status`, `_error_details`
tenta de novo quando o serviço recusa	o argumento `max_retries`, 3 por padrão
retoma do ponto em que parou	o argumento `resume`

Pensar mais, pagar mais

Insper

Tokens que não aparecem na resposta

Na primeira sentença, o modelo gastou 215 tokens de saída, e 153 deles são `_reasoning_tokens`. A resposta tem só três campos curtos.

O que o modelo escreveu nesses 153?

Um rascunho antes da resposta

RESPOSTA

Antes de preencher o formulário, o modelo escreve um raciocínio que não volta para nós: lê o caso, compara opções, confere. O esforço (`low`, `medium`, `high`) diz quanto ele pode escrever ali.

Como uma pessoa: pensar mais costuma dar resposta melhor, mas demora mais. E o rascunho conta como token de saída: se paga.

Três esforços nas dez sentenças

Esforço	Raciocínio	Requerido "e outro"
`low`	525 tokens	copiou "e outro" em 2 das 10
`medium`	2.161 tokens	achou o outro réu nas 2
`high`	3.621 tokens em 6	achou nas 2; 4 linhas caíram no limite da conta gratuita

O mesmo modelo, gpt-oss-120b na Groq, e o mesmo formulário. Com mais esforço, o modelo foi procurar no texto quem era o "outro".

O que se paga

Tokens	O que são
entrada	o pedido e a sentença inteira
saída	a resposta e o raciocínio

O raciocínio não volta para nós, mas é cobrado como saída, que custa mais que a entrada. Na OpenAI, a documentação diz isso com todas as letras.

Quanto custaram as dez

gpt-oss-120b na Groq	Tokens	US$ por milhão
entrada	34.057	0,15
saída, com o raciocínio	2.495	0,60

As dez custariam menos de um centavo de dólar (US$ 0,0066); as 300, uns 20 centavos. Na conta gratuita, a Groq não cobra: limita quantos tokens passam por minuto, e por isso as dez levaram 3 minutos e meio.

Mil decisões: quanto custaria

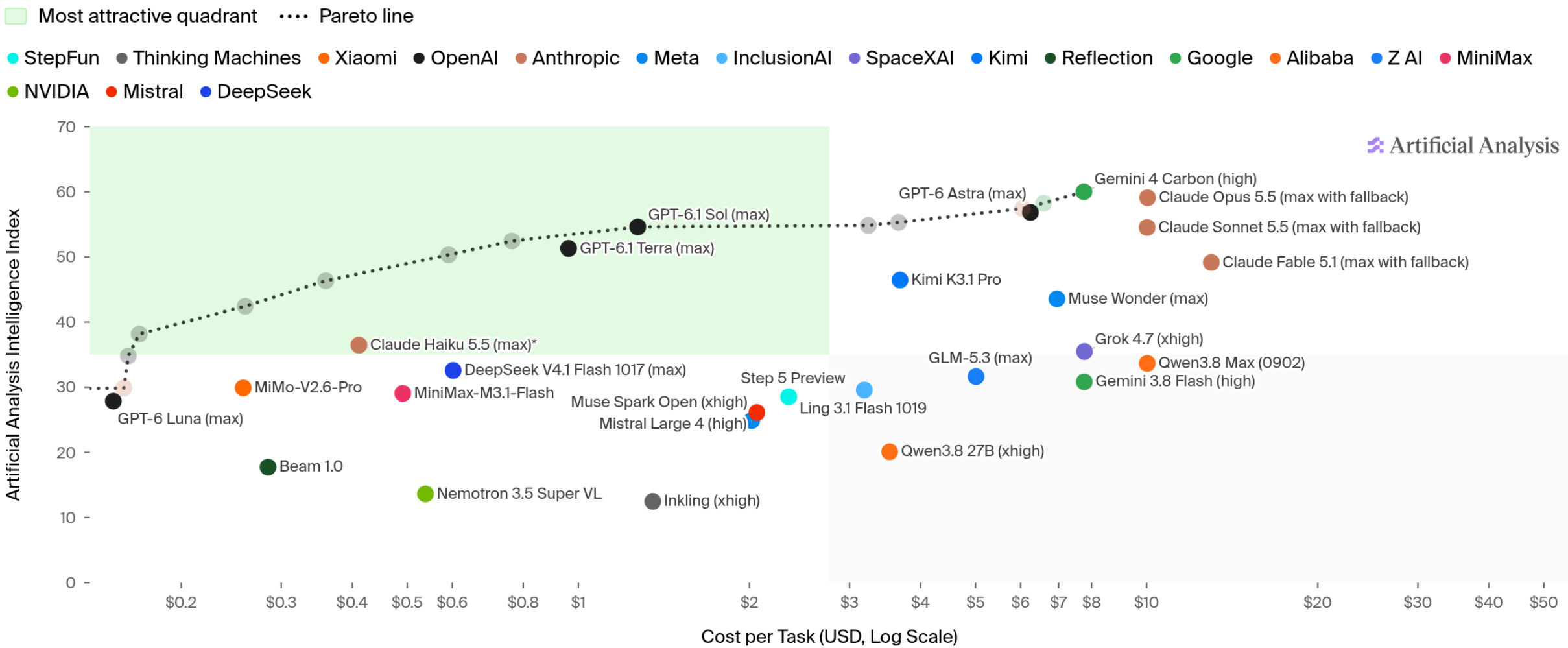
US$ por mil decisões	low	medium	high
gpt-oss-120b, Groq	0,58	0,68	0,91
GPT-6 Luna	0,40	0,48	0,67
GPT-6.1 Terra	3,98	4,79	6,74
GPT-6.1 Sol	7,95	9,58	13,47
GPT-6 Astra	39,76	47,92	67,36

Estimativa com os tokens do gpt-oss nas nossas sentenças. A sentença longa pesa mais que o raciocínio.

Intelligence Index vs. Cost per Task

Artificial Analysis Intelligence Index · Weighted average cost (USD) per Artificial Analysis Intelligence Index task

28 of 723 models



Cost per Intelligence Index Task

A fronteira de hoje

Na fronteira, nenhum modelo mais barato é mais inteligente. Em 04/11/2026 ela vai do GPT-6 Luna, a uns US$ 0,15 por tarefa do teste, ao Gemini 4 Carbon, a uns US$ 8, passando pelo GPT-6.1 Sol e pelo GPT-6 Astra.

Desde 21/10, entraram o Gemini 4 Carbon e o GPT-6 Astra, e os Claude saíram dela. O índice mudou de versão: comparem a ordem, não a nota.

Sem pagar token: Claude Code e Codex

Quem assina o Claude ou o ChatGPT pode usar a cota da assinatura no lugar da chave:

Assinatura	No dataframeit

Claude	`uv add "dataframeit[claude-code]"`
	`provider="claude_code"`
	`model="haiku"`

ChatGPT	`uv add "dataframeit[codex]"`
	`provider="codex"`
	`model="gpt-6-luna"`

Antes da primeira chamada: entrar na conta

O uv add instala a biblioteca, mas não faz o login. Uma vez por computador:

Assinatura	O que falta
Claude	instalar o Claude Code e entrar com a conta
ChatGPT	instalar o Codex e entrar com a conta, com o login guardado em arquivo

```
codex --config cli_auth_credentials_store='"file"' login
```

Travou? Peçam o passo a passo ao próprio Claude ou ChatGPT, colando o erro.

Os dois caminhos lado a lado

Insper

Guardar antes de comparar: `to_csv`

```
dez = decisoes.head(10)
dez["requerido_regex"] = dez["texto"].str.extract(
    r"Requerid[oa]s?:\s*([^\n]+)")
dez.to_csv("dez-decisoes-regex.csv", index=False)
```

`to_csv` grava a tabela num arquivo; `index=False` deixa de fora a numeração das linhas. O caderno gravou `tabela`, a do modelo, do mesmo jeito. Com os dois arquivos salvos, a comparação não depende de rodar tudo de novo.

As mesmas dez, regex contra modelo

```
import pandas as pd

regex = pd.read_csv("dez-decisoes-regex.csv")
modelo = pd.read_csv("dez-decisoes-modelo.csv")

regex["requerido_modelo"] = modelo["requerido"]
print(regex[["requerido_regex", "requerido_modelo"]])
```

`read_csv` relê os dois arquivos. Onde as duas colunas diferem, quem acertou?

Quando usar cada um?

Olhando as dez sentenças: quando vocês usariam a regex, e quando o modelo?

Regra para o que tem forma, modelo para o que tem sentido

RESPOSTA

	Expressão regular	Modelo de linguagem
acerta	em texto de forma fixa	em texto variado
custa	testar a regra	uma chamada por linha
erra	em silêncio	com confiança
se confere	lendo o vazio	lendo uma amostra

Explicar cada campo

Onde o modelo errou?

No cabeçalho	O que o modelo escreveu em `requerido`
Fazenda Pública do Estado de São Paulo e outro	Fazenda Pública do Estado de São Paulo e Município de Marília
MUNICIPIO DE AMERICANA e outro	MUNICIPIO DE AMERICANA
Fazenda Pública do Estado de São Paulo e outro	Fazenda Pública do Estado de São Paulo

O que o formulário deixou de dizer ao modelo?

O campo não diz o que é um réu

RESPOSTA

`requerido: str` aceita qualquer texto. O modelo decidiu sozinho, a cada sentença, se listava os dois réus ou só um, e em que caixa escrevia. Nas sentenças de Americana e de Taquaritinga, o segundo réu está no dispositivo.

O que o modelo precisa saber vai escrito no próprio campo.

Segunda versão: `description` no requerido

```
class Formulario(BaseModel):
    requerido: str = Field(description=(
        "Todos os réus, separados por ' e '. "
        "Quando o cabeçalho diz 'e outro', "
        "procure no texto quem é o outro réu. "
        "Escreva cada nome como na sentença, "
        "com só a primeira letra de cada "
        "palavra maiúscula."))
```

... e no polo passivo

```
    polo_passivo: Literal["público", "privado"] = (
        Field(description=(
        "público se algum réu é ente público: "
        "União, estado, município, Fazenda "
        "ou autarquia; privado se todos são "
        "planos de saúde, operadoras ou "
        "empresas.")))
```

O `resultado` fica como estava: o modelo acertou as dez.

A segunda versão nas mesmas dez

Sentença	O que o modelo escreveu em `requerido`
Americana	Municipio De Americana e Fazenda Publica Do Estado De Sao Paulo
Taquaritinga	Fazenda Pública do Estado de São Paulo e Município de Taquaritinga
Jundiaí	Prefeitura Municipal De Jundiai e Estado De Sao Paulo

O segundo réu apareceu. A regra de maiúsculas virou "Do" e "De", e o acento sumiu em três. Explicar o campo conserta o que é leitura; grafia se padroniza depois, em Python.

Regex ou modelo?

Item 1 de 10

Em cada sentença, extrair o número do processo no padrão CNJ.

R (regex) ou M (modelo)?

Item 1: regex

RESPOSTA

Regex: o número tem formato fixo.

Item 2 de 10

Em cada sentença, dizer se o juiz reconheceu dano moral.

R (regex) ou M (modelo)?

Item 2: modelo

RESPOSTA

Modelo: o juiz diz isso de muitos jeitos, inclusive negando.

Item 3 de 10

Em cada sentença, pegar a data que vem no cabeçalho.

R (regex) ou M (modelo)?

Item 3: regex

RESPOSTA

Regex: o cabeçalho tem lugar e formato fixos.

Item 4 de 10

Em cada sentença, dizer se o atraso do voo foi culpa da companhia ou do tempo.

R (regex) ou M (modelo)?

Item 4: modelo

RESPOSTA

Modelo: o motivo está no meio da fundamentação.

Item 5 de 10

Em cada sentença, achar o valor da causa, escrito como "R$ 10.000,00".

R (regex) ou M (modelo)?

Item 5: regex

RESPOSTA

Regex: o valor tem forma fixa: R$, dígitos, vírgula.

Item 6 de 10

Em cada sentença, dizer se o juiz achou a prova do autor suficiente.

R (regex) ou M (modelo)?

Item 6: modelo

RESPOSTA

Modelo: é um juízo, escrito de muitos jeitos.

Item 7 de 10

Em cada sentença, contar quantas vezes aparece "art. 256".

R (regex) ou M (modelo)?

Item 7: regex

RESPOSTA

Regex: é um texto exato.

Item 8 de 10

Em cada sentença, resumir em uma frase o argumento da defesa.

R (regex) ou M (modelo)?

Item 8: modelo

RESPOSTA

Modelo: resumir é escrever um texto novo.

Item 9 de 10

Em cada sentença, pegar o nome do juiz na linha "Juiz(a) de Direito:".

R (regex) ou M (modelo)?

Item 9: regex

RESPOSTA

Regex: o rótulo é fixo, e o nome vem depois dele.

Item 10 de 10

Em cada sentença, dizer se a companhia ofereceu outro voo ao passageiro.

R (regex) ou M (modelo)?

Item 10: modelo

RESPOSTA

Modelo: a reacomodação aparece no relato, sem fórmula.

Na próxima aula

Um cliente novo: uma grande companhia aérea e o Tema 1.417 do STF. Desta vez, a base não vem pronta.